```python
a =float(input("Enter first number: "))
b = float(input("Enter second number: "))
op = input("Enter operator (+, -, *, /): ")

if op == "+":
    print("Result:", a + b)
elif op == "-":
    print("Result:", a - b)
elif op == "*":
    print("Result:", a * b)
elif op == "/":
    print("Result:", a / b)
else:
    print("Invalid operator")
```

```
Enter first number:  6
Enter second number:  5
Enter operator (+, -, *, /):  +
Result: 11.0
```

```python
c = float(input("Enter temperature in Celsius: "))
f = (c * 9/5) + 32
print("Temperature in Fahrenheit:", f)
```

```
Enter temperature in Celsius:  65
Temperature in Fahrenheit: 149.0
```

```python
f = float(input("Enter temperature in Fahrenheit: "))
c = (f - 32) * 5/9
print("Temperature in Celsius:", c)
```

```
Enter temperature in Fahrenheit:  56
Temperature in Celsius: 13.333333333333334
```

```python
a = float(input("Enter first number: "))
b = float(input("Enter second number: "))
c = float(input("Enter third number: "))

print("Largest number is:", max(a, b, c))
```

```
Enter first number:  5
Enter second number:  6
Enter third number:  11
Largest number is: 11.0
```

```python
a = float(input("Enter first number: "))
b = float(input("Enter second number: "))
c = float(input("Enter third number: "))

print("Smallest number is:", min(a, b, c))
```

```
Enter first number:  7
Enter second number:  8
Enter third number:  9
Smallest number is: 7.0
```

```python
def factorial(n):
    if n < 0:
        return "Factorial is not defined for negative numbers"
    elif n == 0 or n == 1:
        return 1
    else:
        return n * factorial(n - 1)

num = int(input("Enter a number: "))

print("Factorial is:", factorial(num))
```

```
Enter a number:  11
Factorial is: 39916800
```

```python
sentence = "Hello"
word_count = len(sentence.split())
print(word_count)
```

```
1
```

```python
def fib(n):
    return n if n <= 1 else fib(n-1) + fib(n-2)

n = int(input("Enter a number: "))
print(fib(n))
```

```
Enter a number:  11
89
```

```
[1]: a=[10,20,30,40]

[2]: print(a)
     [10, 20, 30, 40]

[6]: a.append(10)

[7]: a

[7]: [10, 20, 30, 40, 100, 10, 10]

[8]: c=a.copy()

[9]: c

[9]: [10, 20, 30, 40, 100, 10, 10]

[14]: a.count(20)

[14]: 1

[17]: a.extend([1,2,3])

[18]: a

[18]: [10, 20, 30, 40, 100, 10, 10, 1, 2, 3]

[19]: a.index(30)

[19]: 2
```

```
[30]: a.insert(5,11)
      a

[30]: [40, 30, 20, 11, 11, 11, 11, 10, 10, 10, 2, 1]

[22]: a.pop()

[22]: 3

[23]: a

[23]: [10, 20, 30, 40, 100, 11, 11, 10, 10, 1, 2]

[25]: a.remove(100)
      a

[25]: [10, 20, 30, 40, 11, 11, 10, 10, 1, 2]

[26]: a.sort()

[27]: a

[27]: [1, 2, 10, 10, 10, 11, 11, 20, 30, 40]

[28]: a.reverse()
      a

[28]: [40, 30, 20, 11, 11, 10, 10, 10, 2, 1]

[31]: a.insert(5,6)
      a

[31]: [40, 30, 20, 11, 11, 6, 11, 11, 10, 10, 10, 2, 1]
```

```
a.clear()
a
```

```
[]
```

```
a=[5,11,74,63]
a
```

```
[5, 11, 74, 63]
```

```
print ("lenght=",len(a))
```

```
lenght= 4
```

```
print (a[:])
```

```
[5, 11, 74, 63]
```

```
a[1:4]
```

```
[11, 74, 63]
```

```
a[0:3]
```

```
[5, 11, 74]
```

```
a[:2]
```

```
[5, 11]
```

```
a[0:4:2]
```

```
[5, 74]
```

```
[50]: a[-5:-1]
```

```
[50]: [5, 11, 74]
```

```
[52]: a[-1:-4]
```

```
[52]: []
```

```
[53]: c=(10,20,30,40,11,5)
```

```
[54]: c=(10,20,30,"anu",11,5)
      c
```

```
[54]: (10, 20, 30, 'anu', 11, 5)
```

```
[56]: c=(5,11)
      c
```

```
[56]: (5, 11)
```

```
[57]: c.count(11)
```

```
[57]: 1
```

```
[65]: c
```

```
[65]: (5, 11)
```

```
[66]: c.index(5)
```

```
[66]: 0
```

[68]:
```
c=(10,20,30,"anu",11,5)
c
```

[68]:
```
(10, 20, 30, 'anu', 11, 5)
```

[70]:
```
c.index(11)
```

[70]:
```
4
```

[71]:
```
len(c)
```

[71]:
```
6
```

```
[1]: a={1,2,3,4}
     a
```

```
[1]: {1, 2, 3, 4}
```

```
[2]: b={3,4,5,6}
     b
```

```
[2]: {3, 4, 5, 6}
```

```
[4]: a.add(7)
     a
```

```
[4]: {1, 2, 3, 4, 7}
```

```
[5]: c=a.copy()
     c
```

```
[5]: {1, 2, 3, 4, 7}
```

```
[6]: a
```

```
[6]: {1, 2, 3, 4, 7}
```

```
[7]: a.union(b)
```

```
[7]: {1, 2, 3, 4, 5, 6, 7}
```

```
[9]: a.intersection(b)
```

```
[9]: {3, 4}
```

```
[10]: a.difference(b)
```

```
[10]: {1, 2, 7}
```

```
[11]: b.difference(a)
```

```
[11]: {5, 6}
```

[13]:
```
a.symmetric_difference(b)
```

[13]:
```
{1, 2, 5, 6, 7}
```

[18]:
```
a.discard(2)
a
```

[18]:
```
{1, 3, 4, 7, 20, 30}
```

[19]:
```
a.update([20,30])
a
```

[19]:
```
{1, 3, 4, 7, 20, 30}
```

```
a.difference_update
```

[20]:
```
a
```

[20]:
```
{1, 3, 4, 7, 20, 30}
```

[21]:
```
b
```

[21]:
```
{3, 4, 5, 6}
```

[22]:
```
a.isdisjoint(b)
```

[22]:
```
False
```

[23]:
```
a.issubset(b)
```

[23]:
```
False
```

[24]:
```
a.issuperset(b)
```

[24]:
```
False
```

[32]:
```
def list_operation():
    my_list = []

    while True:
        print("\nList Operation:")
        print("1. Insert an element")
        print("2. Delete an element")
        print("3. Find an element")
        print("4. Display the list")
        print("5. Exit")

        choice = int(input("Enter your choice: "))

        if choice == 1:
            element = input("Enter element to insert: ")
            my_list.append(element)
            print(f"Element '{element}' inserted")

        elif choice == 2:
            element = input("Enter element to delete: ")
            if element in my_list:
                my_list.remove(element)
                print(f"Element '{element}' deleted")
            else:
                print(f"Element '{element}' not found")

        elif choice == 3:
            element = input("Enter element to find: ")
```

```python
        elif choice == 3:
            element = input("Enter element to find: ")
            if element in my_list:
                print(f"Element '{element}' found")
            else:
                print(f"Element '{element}' not found")

        elif choice == 4:
            print(f"Current list: {my_list}")

        elif choice == 5:
            print("Exiting program...")
            break

        else:
            print("Invalid choice, please try again")


list_operation()
```

```
List Operation:
1. Insert an element
2. Delete an element
3. Find an element
4. Display the list
5. Exit
Enter your choice:  1
Enter element to insert:  11
Element '11' inserted
```

```
List Operation:
1. Insert an element
2. Delete an element
3. Find an element
4. Display the list
5. Exit
Enter your choice:  2
Enter element to delete:  12
Element '12' not found

List Operation:
1. Insert an element
2. Delete an element
3. Find an element
4. Display the list
5. Exit
Enter your choice:  5
Exiting program...
```

```
[1]: a={1,2,3,4}
     a
```

```
[1]: {1, 2, 3, 4}
```

```
[2]: b={3,4,5,6}
     b
```

```
[2]: {3, 4, 5, 6}
```

```
[4]: a.add(7)
     a
```

```
[4]: {1, 2, 3, 4, 7}
```

```
[5]: c=a.copy()
     c
```

```
[5]: {1, 2, 3, 4, 7}
```

```
[6]: a
```

```
[6]: {1, 2, 3, 4, 7}
```

```
[7]: a.union(b)
```

```
[7]: {1, 2, 3, 4, 5, 6, 7}
```

```
[9]: a.intersection(b)
```

```
[9]: {3, 4}
```

```
[10]: a.difference(b)
```

```
[10]: {1, 2, 7}
```

```
[11]: b.difference(a)
```

```
[11]: {5, 6}
```

[13]:
```
a.symmetric_difference(b)
```

[13]:
```
{1, 2, 5, 6, 7}
```

[18]:
```
a.discard(2)
a
```

[18]:
```
{1, 3, 4, 7, 20, 30}
```

[19]:
```
a.update([20,30])
a
```

[19]:
```
{1, 3, 4, 7, 20, 30}
```

```
a.difference_update
```

[20]:
```
a
```

[20]:
```
{1, 3, 4, 7, 20, 30}
```

[21]:
```
b
```

[21]:
```
{3, 4, 5, 6}
```

[22]:
```
a.isdisjoint(b)
```

[22]:
```
False
```

[23]:
```
a.issubset(b)
```

[23]:
```
False
```

[24]:
```
a.issuperset(b)
```

[24]:
```
False
```

[32]:
```
def list_operation():
    my_list = []

    while True:
        print("\nList Operation:")
        print("1. Insert an element")
        print("2. Delete an element")
        print("3. Find an element")
        print("4. Display the list")
        print("5. Exit")

        choice = int(input("Enter your choice: "))

        if choice == 1:
            element = input("Enter element to insert: ")
            my_list.append(element)
            print(f"Element '{element}' inserted")

        elif choice == 2:
            element = input("Enter element to delete: ")
            if element in my_list:
                my_list.remove(element)
                print(f"Element '{element}' deleted")
            else:
                print(f"Element '{element}' not found")

        elif choice == 3:
            element = input("Enter element to find: ")
```

```python
        elif choice == 3:
            element = input("Enter element to find: ")
            if element in my_list:
                print(f"Element '{element}' found")
            else:
                print(f"Element '{element}' not found")

        elif choice == 4:
            print(f"Current list: {my_list}")

        elif choice == 5:
            print("Exiting program...")
            break

        else:
            print("Invalid choice, please try again")


list_operation()
```

```
List Operation:
1. Insert an element
2. Delete an element
3. Find an element
4. Display the list
5. Exit
Enter your choice:  1
Enter element to insert:  11
Element '11' inserted
```

```
List Operation:
1. Insert an element
2. Delete an element
3. Find an element
4. Display the list
5. Exit
Enter your choice:  2
Enter element to delete:  12
Element '12' not found

List Operation:
1. Insert an element
2. Delete an element
3. Find an element
4. Display the list
5. Exit
Enter your choice:  5
Exiting program...
```

jupyter Lab 4 Last Checkpoint: 7 minutes ago

File Edit View Run Kernel Settings Help Trusted

Code JupyterLab Python 3 (ipykernel) Anaconda Toolbox

[5]:
```
d1={"name":'navya',"class":'b section'}
d2={"course":'bca'}
d3=d1|d2
print(d3)
```

```
{'name': 'navya', 'class': 'b section', 'course': 'bca'}
```

File Edit View Run Kernel Settings Help

Code

[7]:
```
import numpy as np
a1=np.array([1,2,3,4,5,6,7,8,9])
print(a1)
rsa=a1.reshape(3,3)
print(rsa)
# Accessing elements using indexing
print("accessing ")
print(rsa[2][1])
print(rsa[0:2])

print("all row 2nd column")
print(rsa[:,2])

print("modify")
rsa[1][1]=55

print("\n")
sum=np.sum(rsa)
print(sum)

print("sum of column")
sumc=np.sum(rsa,axis=0)
print(sumc)

print("sum of row")
sumr=np.sum(rsa,axis=1)
print(sumr)
```

```
[1 2 3 4 5 6 7 8 9]
[[1 2 3]
 [4 5 6]
 [7 8 9]]
accessing
```

```
accessing
8
[[1 2 3]
 [4 5 6]]
all row 2nd column
[3 6 9]
modify


95
sum of column
[12 65 18]
sum of row
[ 6 65 24]
```

[8]:
```
a=np.array([1,2,3])
print(a+2)
print(a*3)
```

```
[3 4 5]
[3 6 9]
```

[9]:
```
a=[1,6,5]
print(np.sum(a))
print(np.mean(a))
print(np.median(a))
print(np.max(a))
print(np.min(a))
```

```
12
4.0
5.0
6
1
```

[14]:
```
a=np.array([1,2,3])
a=np.append(a,4)
print(a)
a=np.delete(a,1)
a
```

```
[1 2 3 4]
```

[14]:
```
array([1, 3, 4])
```

[19]:
```
a = np.array([1, 4, 6, 2])
print(np.sort(a))

(np.zeros((2, 3)))
(np.ones((2, 3)))
(np.eye(3))
(np.arange(0, 10, 2))
(np.linspace(0, 1, 3))
```

```
[1 2 4 6]
```

[19]:
```
array([0. , 0.5, 1. ])
```

1

```
[8]: import numpy as np
m1=[[1,2,3],[4,5,6]]
m2=[[9,8,7],[6,5,4]]
m=np.add(m1,m2)
print(m)
```

```
[[10 10 10]
 [10 10 10]]
```

2

```
[10]: import numpy as np
m1=[[1,2,3],[4,5,6]]
m2=[[9,8,7],[6,5,4]]
m=np.subtract(m1,m2)
print(m)
```

```
[[-8 -6 -4]
 [-2  0  2]]
```

3

```
[12]: import numpy as np
m1=[[1,2,3],[4,5,6]]
m2=[[9,8,7],[6,5,4]]
m=np.multiply(m1,m2)
print(m)
```

```
[[ 9 16 21]
 [24 25 24]]
```

4

```
[19]: import numpy as np
m1=[[1,2,3],[4,5,6]]
m2=[[9,8,7],[6,5,4],[1,2,3]]
m=np.dot(m1,m2)
print(m)
```

```
[[24 24 24]
 [72 69 66]]
```

5

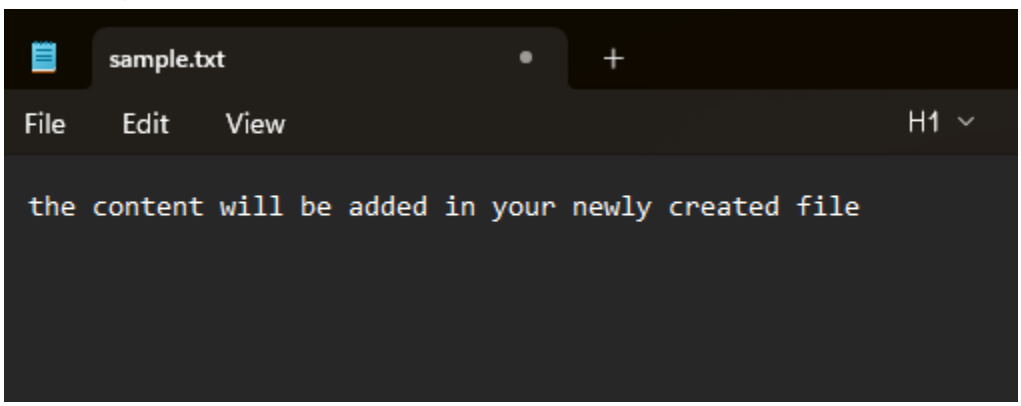
```
[14]: import numpy as np
m1=[[1,2,3],[4,5,6]]
m2=[[9,8,7],[6,5,4]]
m=np.transpose(m1)
print(m)
```

```
[[1 4]
 [2 5]
 [3 6]]
```

6

```
[30]: file1=open("C:\\Users\\RVUW280\\Desktop\\sample.txt","x")
file1.write("the content will be added in your newly created file")
file1.close()
```

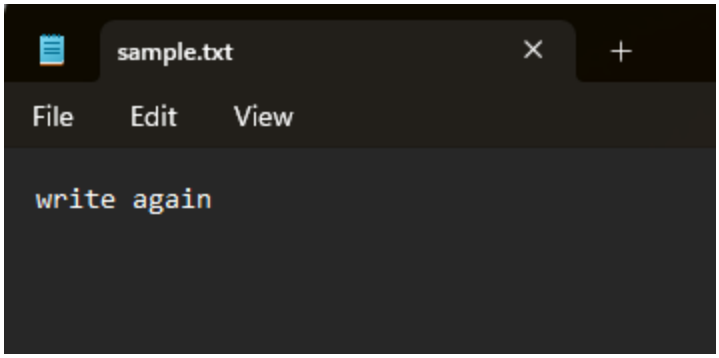
Output



7

```
[34]: file=open("C:\\Users\\RVUW280\\Desktop\\sample.txt","w")
file.write("write again")
file.close()
```

Output



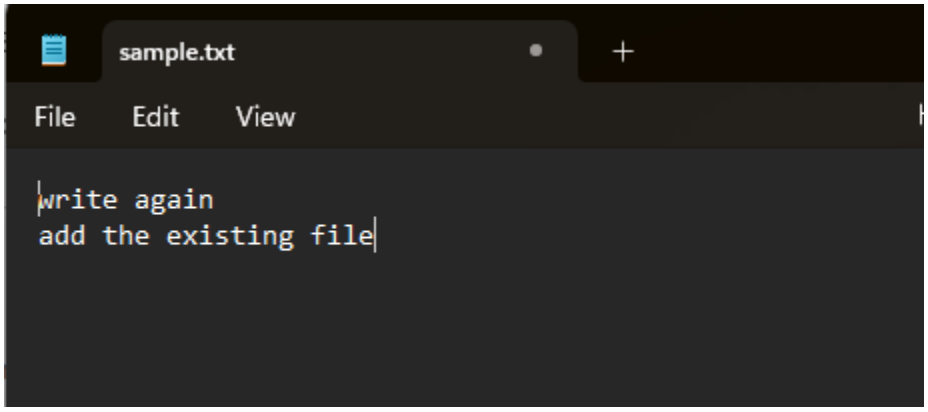
8

```
[39]: f=open("C:\\Users\\RVUW280\\Desktop\\sample.txt","r")
      res=f.read()
      print(res)

      write again
```

9

```
[42]: f1=open("C:\\Users\\RVUW280\\Desktop\\sample.txt","a")
      f1.write("\n add the existing file")
      f1.close()
```

Output



10

```
[43]: with open("C:\\Users\\RVUW280\\Desktop\\sample.txt","r") as file:
          print(file.read())
```

```
write again
 add the exisiting file
 add the existing file
```

11.

```
[44]: with open("C:\\Users\\RVUW280\\Desktop\\sample1.txt","x") as file:
          print(file.write("hello"))
```

```
5
```

1.

```
[3]: try:
         a=10
         b=0
         print(a/b)
     except:
         print("error occured")
```

```
error occured
```

2.

```
[5]: try:
         num=int(input("enter a number:"))
         print(num)
     except valueerror:
         print("invalid input ! please enter a number:")
```

```
enter a number: 65
65
```

3.

```
[11]: try:
          a=int(input("enter a number:"))
          b=int(input("enter a number:"))
          print(a/b)
      except Valueerror:
          print("invalid number entered")
      except zeroDivisionError:
          print("cannot divide by zero")
```

```
enter a number: 11
enter a number: 65
0.16923076923076924
```

4.

```
[17]: try:
          a=10
          b=2
          result=a/b
      except valueerror:
          print("divide by zero not allowed ")
      else:
          print("result",result)
```

```
result 5.0
```

5.

```
[14]: try:
          f=open("data.txt","r")
          print(f.read())
      except FileNotFoundError:
          print("File Not Found")
      finally:
          print("execution completed")
```

```
File Not Found
execution completed
```

6.

```
[19]: try:
          num=int("abc")
      except (ValueError,TypeError):
          print("Error occured while converting value")
```

```
Error occured while converting value
```

7.

```
[40]: try:
        age=-5
        if age<0:
          raise ValueError("age cannot be negative")
        print("age is:", age)
      except ValueError as e:
        print(e)
```

```
age cannot be negative
```

8.

```
[46]: class MyError(Exception):
          pass

      try:
          raise MyError("custom error occurred")
      except MyError as e:
          print(e)
```

```
custom error occurred
```

9.

```
[47]: try:
          file=open("sample.txt","r")
          content=file.read()
          print(content)
      except FileNotFoundError:
          print("file does not exist")
```

```
file does not exist
```

10.

```
[49]: try:
          x=5/0
      except Exception as e:
          print("error message:",e)
```

```
error message: division by zero
```

1.

```
[4]: import pandas as pd
     data=pd.Series([10,20,30,40])
     print(data)
```

```
0    10
1    20
2    30
3    40
dtype: int64
```

2.

```
[5]: import pandas as pd
     data=pd.Series([10,20,30,40], index=["a","b","c","d"])
     print(data)
```

```
a    10
b    20
c    30
d    40
dtype: int64
```

3.

```
[8]: import pandas as pd
     data={
         "name":["ram","vani"],
         "age":[20,22]
     }
     df=pd.DataFrame(data)
     print(df)
```

```
   name  age
0   ram   20
1  vani   22
```

4.

```
[18]: import pandas as pd
data = {
    "Name": ["Ram", "Vani"],
    "Age": [20,25 ]
}

df = pd.DataFrame(data)
print(df)
print("\n")
print(df.Name)
print("\n")
print(df.loc[0])
print("\n")
print(df.loc[1])
```

```
   Name  Age
0   Ram   20
1  Vani   25


0     Ram
1    Vani
Name: Name, dtype: object


Name    Ram
Age      20
Name: 0, dtype: object


Name    Vani
Age       25
Name: 1, dtype: object
```

5.

```
[22]: df=pd.read_csv("C:\\Users\\RVUW280\\Desktop\\ABC.csv")
print(df)
```

```
  name department  marks
0  anu        bca   87.0
1  abc        mca   90.0
2  xyz        mca    NaN
3  mnl        BSC   54.0
```

6.

```
[23]: df.isnull()
```

[23]:

	name	department	marks
0	False	False	False
1	False	False	False
2	False	False	True
3	False	False	False

7.

```
[24]: df.dropna()  # remove the missing data      # NaN - Numpy , Python-None , Pandas-pd.Na
```

[24]:

	name	department	marks
0	anu	bca	87.0
1	abc	mca	90.0
3	mnl	BSC	54.0

8.

```
[27]: df.fillna(0)
```

[27]:

	name	department	marks
0	anu	bca	87.0
1	abc	mca	90.0
2	xyz	mca	0.0
3	mnl	BSC	54.0

9.

```
[32]: # basic statistics method
print(df["marks"].mean())
print(df.marks.sum())  # use this type also
```

```
77.0
231.0
```

```
[31]: print(df.marks.min())
print(df.marks.max())
print(df.marks.count())
```

```
54.0
90.0
3
```

10.

```
[38]: df.shape
```

```
[38]: (4, 3)
```

11.

```
[40]: df.ndim
```

```
[40]: 2
```

12.

```
[41]: len(df)
```

```
[41]: 4
```

13..

```
[42]: df.head(1)
```

[42]:

	name	department	marks
0	anu	bca	87.0

14.

```
[43]: df.tail(1)
```

[43]:

	name	department	marks
3	mnl	BSC	54.0

15.

```
[44]: df.index
```

```
[44]: RangeIndex(start=0, stop=4, step=1)
```

16.

```
[45]: df.columns
```

```
[45]: Index(['name', 'department', 'marks'], dtype='object')
```

17.

```
[49]: import pandas as pd

# Create table data
data = {
    "ID": [1, 2, 3, 4],
    "Name": ["Rahul", "Priya", "Amit", "Sara"],
    "Marks": [85, 90, 78, 92]
}
# Convert to table (DataFrame)
df = pd.DataFrame(data)

# Save as CSV file
df.to_csv("students.csv",index=False)

print("CSV file created ")
```

```
CSV file created
```

1. Write a Python program that creates a Pandas DataFrame from scratch using a dictionary .Display the DataFrame and then add a new column with calculated values.

[6]:
```
import pandas as pd
data = { 'Name':['Alice','Bob','Charlie','David'],
         'Age':[25,30,3,40],
         'Salary':[50000,60000,70000,80000]}
df=pd.DataFrame(data)
print("DataFrame from Scratch:")
print(df)
df['Bonus']=df['Salary']*0.10
print("\nDataFrame after adding 'Bonus'column:")
print(df)
```

```
DataFrame from Scratch:
      Name  Age  Salary
0    Alice   25   50000
1      Bob   30   60000
2  Charlie    3   70000
3    David   40   80000

DataFrame after adding 'Bonus'column:
      Name  Age  Salary   Bonus
0    Alice   25   50000  5000.0
1      Bob   30   60000  6000.0
2  Charlie    3   70000  7000.0
3    David   40   80000  8000.0
```

2. Write a Python program to load a CSV file into a Pandas DataFrame, display the first 6 rows , and show basic information such as column names and data types.

```
[9]: import pandas as pd
df=pd.read_csv("C:\\Users\\RVUW280\\Downloads\\growth rate - Sheet1.csv")
print("First 6 rows of the DataFrame:")
print(df.head(6))
print("\nColumn names and data types:")
print(df.info())
```

```
First 6 rows of the DataFrame:
   Year  McDonalds_Revenue_$Billion  Growth_rate_percent   Q1   Q2   Q3   Q4
0  1999                        13.3                  NaN  NaN  NaN  NaN  NaN
1  2000                        14.2                  7.0  NaN  NaN  NaN  NaN
2  2001                        14.9                  4.0  NaN  NaN  NaN  NaN
3  2002                        15.4                  4.0  NaN  NaN  NaN  3.0
4  2003                        17.1                 11.0  3.8  4.3  4.5  4.6
5  2004                        18.6                  8.0  4.4  4.7  4.9  4.5

Column names and data types:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 24 entries, 0 to 23
Data columns (total 7 columns):
 #   Column                      Non-Null Count  Dtype
---  ------                      --------------  -----
 0   Year                        24 non-null     int64
 1   McDonalds_Revenue_$Billion  24 non-null     float64
 2   Growth_rate_percent         23 non-null     float64
 3   Q1                          20 non-null     float64
 4   Q2                          20 non-null     float64
 5   Q3                          20 non-null     float64
 6   Q4                          21 non-null     float64
dtypes: float64(6), int64(1)
memory usage: 1.4 KB
None
```

5]:
```python
import pandas as pd
df=pd.read_csv("C:\\Users\\RVUW280\\Downloads\\growth rate - Sheet1.csv")
print(df)
```

```
    Year  McDonalds_Revenue_$Billion  Growth_rate_percent   Q1   Q2   Q3   Q4
0   1999                        13.3                  NaN  NaN  NaN  NaN  NaN
1   2000                        14.2                  7.0  NaN  NaN  NaN  NaN
2   2001                        14.9                  4.0  NaN  NaN  NaN  NaN
3   2002                        15.4                  4.0  NaN  NaN  NaN  3.0
4   2003                        17.1                 11.0  3.8  4.3  4.5  4.6
5   2004                        18.6                  8.0  4.4  4.7  4.9  4.5
6   2005                        19.1                  3.0  4.8  5.1  5.3  3.9
7   2006                        20.9                  9.0  4.9  5.4  5.5  5.1
8   2007                        22.8                  9.0  5.3  5.8  5.9  5.8
9   2008                        23.5                  3.0  5.6  6.1  6.3  5.6
10  2009                        22.7                 -3.0  5.1  5.6  6.0  6.0
11  2010                        24.1                  6.0  5.6  5.9  6.3  6.2
12  2011                        27.0                 12.0  6.1  6.9  7.2  6.8
13  2012                        27.6                  2.0  6.5  6.9  7.2  7.0
14  2013                        28.1                  2.0  6.6  7.1  7.3  7.1
15  2014                        27.4                 -2.0  6.7  7.2  7.0  6.6
16  2015                        25.4                 -7.0  6.0  6.5  6.6  6.3
17  2016                        24.6                 -3.0  5.9  6.3  6.4  6.0
18  2017                        22.8                 -7.0  5.7  6.0  5.8  5.3
19  2018                        21.3                 -7.0  5.1  5.4  5.4  5.4
20  2019                        21.4                  1.0  5.0  5.3  5.6  5.4
21  2020                        19.2                -10.0  4.7  3.8  5.4  5.3
22  2021                        23.2                 21.0  5.1  5.9  6.2  6.0
23  2022                        23.2                  0.0  5.7  5.7  5.9  5.9
```

]:
```python
print("\nMissing values in the DataFrame:")
print(df.isnull().sum())
```

```
Missing values in the DataFrame:
Year                          0
McDonalds_Revenue_$Billion    0
Growth_rate_percent           1
Q1                            4
Q2                            4
Q3                            4
Q4                            3
dtype: int64
```

```python
df.Q1=df['Q1'].fillna(df['Q1'].mean())
print(df.Q1)
```

```
0     5.43
1     5.43
2     5.43
3     5.43
4     3.80
5     4.40
6     4.80
7     4.90
8     5.30
9     5.60
10    5.10
11    5.60
12    6.10
13    6.50
14    6.60
15    6.70
16    6.00
17    5.90
18    5.70
19    5.10
20    5.00
21    4.70
22    5.10
23    5.70
Name: Q1, dtype: float64
```

[21]:
```
import pandas as pd
df=pd.read_csv("C:\\Users\\RVUW280\\Downloads\\N1 - Sheet1
print(df.YearsExperience.mean())
```

```
2.266666666666667
```

[22]:
```
print(df.YearsExperience.median())
```

```
2.2
```

[23]:
```
print(df.YearsExperience.mode())
```

```
0    3.2
Name: YearsExperience, dtype: float64
```

[24]:
```
print(df.YearsExperience.min())
```

```
1.1
```

[25]:
```
print(df.YearsExperience.max())
```

```
3.2
```

[26]:
```
print(df.YearsExperience.var())
```

```
0.705
```

[27]:
```
print(df.YearsExperience.std())
```

```
0.8396427811873333
```

1. Write a Python program that creates a Pandas DataFrame from scratch using a dictionary .Display the DataFrame and then add a new column with calculated values.

[6]:
```
import pandas as pd
data = { 'Name':['Alice','Bob','Charlie','David'],
         'Age':[25,30,3,40],
         'Salary':[50000,60000,70000,80000]}
df=pd.DataFrame(data)
print("DataFrame from Scratch:")
print(df)
df['Bonus']=df['Salary']*0.10
print("\nDataFrame after adding 'Bonus'column:")
print(df)
```

```
DataFrame from Scratch:
      Name  Age  Salary
0    Alice   25   50000
1      Bob   30   60000
2  Charlie    3   70000
3    David   40   80000

DataFrame after adding 'Bonus'column:
      Name  Age  Salary   Bonus
0    Alice   25   50000  5000.0
1      Bob   30   60000  6000.0
2  Charlie    3   70000  7000.0
3    David   40   80000  8000.0
```

2. Write a Python program to load a CSV file into a Pandas DataFrame, display the first 6 rows , and show basic information such as column names and data types.

```
[9]: import pandas as pd
df=pd.read_csv("C:\\Users\\RVUW280\\Downloads\\growth rate - Sheet1.csv")
print("First 6 rows of the DataFrame:")
print(df.head(6))
print("\nColumn names and data types:")
print(df.info())
```

```
First 6 rows of the DataFrame:
   Year  McDonalds_Revenue_$Billion  Growth_rate_percent   Q1   Q2   Q3   Q4
0  1999                        13.3                  NaN  NaN  NaN  NaN  NaN
1  2000                        14.2                  7.0  NaN  NaN  NaN  NaN
2  2001                        14.9                  4.0  NaN  NaN  NaN  NaN
3  2002                        15.4                  4.0  NaN  NaN  NaN  3.0
4  2003                        17.1                 11.0  3.8  4.3  4.5  4.6
5  2004                        18.6                  8.0  4.4  4.7  4.9  4.5

Column names and data types:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 24 entries, 0 to 23
Data columns (total 7 columns):
 #   Column                      Non-Null Count  Dtype
---  ------                      --------------  -----
 0   Year                        24 non-null     int64
 1   McDonalds_Revenue_$Billion  24 non-null     float64
 2   Growth_rate_percent         23 non-null     float64
 3   Q1                          20 non-null     float64
 4   Q2                          20 non-null     float64
 5   Q3                          20 non-null     float64
 6   Q4                          21 non-null     float64
dtypes: float64(6), int64(1)
memory usage: 1.4 KB
None
```

5]:
```python
import pandas as pd
df=pd.read_csv("C:\\Users\\RVUW280\\Downloads\\growth rate - Sheet1.csv")
print(df)
```

```
    Year  McDonalds_Revenue_$Billion  Growth_rate_percent   Q1   Q2   Q3   Q4
0   1999                        13.3                  NaN  NaN  NaN  NaN  NaN
1   2000                        14.2                  7.0  NaN  NaN  NaN  NaN
2   2001                        14.9                  4.0  NaN  NaN  NaN  NaN
3   2002                        15.4                  4.0  NaN  NaN  NaN  3.0
4   2003                        17.1                 11.0  3.8  4.3  4.5  4.6
5   2004                        18.6                  8.0  4.4  4.7  4.9  4.5
6   2005                        19.1                  3.0  4.8  5.1  5.3  3.9
7   2006                        20.9                  9.0  4.9  5.4  5.5  5.1
8   2007                        22.8                  9.0  5.3  5.8  5.9  5.8
9   2008                        23.5                  3.0  5.6  6.1  6.3  5.6
10  2009                        22.7                 -3.0  5.1  5.6  6.0  6.0
11  2010                        24.1                  6.0  5.6  5.9  6.3  6.2
12  2011                        27.0                 12.0  6.1  6.9  7.2  6.8
13  2012                        27.6                  2.0  6.5  6.9  7.2  7.0
14  2013                        28.1                  2.0  6.6  7.1  7.3  7.1
15  2014                        27.4                 -2.0  6.7  7.2  7.0  6.6
16  2015                        25.4                 -7.0  6.0  6.5  6.6  6.3
17  2016                        24.6                 -3.0  5.9  6.3  6.4  6.0
18  2017                        22.8                 -7.0  5.7  6.0  5.8  5.3
19  2018                        21.3                 -7.0  5.1  5.4  5.4  5.4
20  2019                        21.4                  1.0  5.0  5.3  5.6  5.4
21  2020                        19.2                -10.0  4.7  3.8  5.4  5.3
22  2021                        23.2                 21.0  5.1  5.9  6.2  6.0
23  2022                        23.2                  0.0  5.7  5.7  5.9  5.9
```

]:
```python
print("\nMissing values in the DataFrame:")
print(df.isnull().sum())
```

```
Missing values in the DataFrame:
Year                          0
McDonalds_Revenue_$Billion    0
Growth_rate_percent           1
Q1                            4
Q2                            4
Q3                            4
Q4                            3
dtype: int64
```

```python
df.Q1=df['Q1'].fillna(df['Q1'].mean())
print(df.Q1)
```

```
0     5.43
1     5.43
2     5.43
3     5.43
4     3.80
5     4.40
6     4.80
7     4.90
8     5.30
9     5.60
10    5.10
11    5.60
12    6.10
13    6.50
14    6.60
15    6.70
16    6.00
17    5.90
18    5.70
19    5.10
20    5.00
21    4.70
22    5.10
23    5.70
Name: Q1, dtype: float64
```

[21]:
```
import pandas as pd
df=pd.read_csv("C:\\Users\\RVUW280\\Downloads\\N1 - Sheet1
print(df.YearsExperience.mean())
```

```
2.266666666666667
```

[22]:
```
print(df.YearsExperience.median())
```

```
2.2
```

[23]:
```
print(df.YearsExperience.mode())
```

```
0    3.2
Name: YearsExperience, dtype: float64
```

[24]:
```
print(df.YearsExperience.min())
```

```
1.1
```

[25]:
```
print(df.YearsExperience.max())
```

```
3.2
```

[26]:
```
print(df.YearsExperience.var())
```

```
0.705
```

[27]:
```
print(df.YearsExperience.std())
```

```
0.8396427811873333
```

[12]:
```python
import os
os.chdir("C:\\Users\\RVUW280\\Downloads")

import pandas as pd
df=pd.read_csv("experience  - Sheet1.csv")
print(df)
print(df.describe())
print(df.quantile([0.25,0.5,0.75]))

# Calculating skewness and kurtosis
skewness = df['YearsExperience'].skew()
kurtosis = df['YearsExperience'].kurt()
print(df.skew())
print(df.kurt())
```

```
    YearsExperience  Salary
0               1.1   39343
1               1.3   46205
2               1.5   37731
3               2.0   43525
4               2.2   39891
5               2.9   56642
6               3.0   60150
7               3.2   54445
8               3.2   64445
9               3.7   57189
10              3.9   63218
11              4.0   55794
12              4.0   56957
13              4.1   57081
14              4.5   61111
15              4.9   67938
16              5.1   66029
17              5.3   83088
```

```
18              5.9   81363
19              6.0   93940
20              6.8   91738
21              7.1   98273
22              7.9  101302
23              8.2  113812
24              8.7  109431
25              9.0  105582
26              9.5  116969
27              9.6  112635
28             10.3  122391
29             10.5  121872
       YearsExperience         Salary
count        30.000000      30.000000
mean          5.313333   76003.000000
std           2.837888   27414.429785
min           1.100000   37731.000000
25%           3.200000   56720.750000
50%           4.700000   65237.000000
75%           7.700000  100544.750000
max          10.500000  122391.000000
      YearsExperience     Salary
0.25              3.2   56720.75
0.50              4.7   65237.00
0.75              7.7  100544.75
YearsExperience    0.37956
Salary             0.35412
dtype: float64
YearsExperience   -1.012212
Salary            -1.295421
dtype: float64
```

[6]:
```python
import os
os.chdir("C:\\Users\\RVUW280\\Downloads")

import pandas as pd
# Load the CSV file
df=pd.read_csv("experience  - Sheet1.csv")
print(df)
```

```
    YearsExperience  Salary
0               1.1   39343
1               1.3   46205
2               1.5   37731
3               2.0   43525
4               2.2   39891
5               2.9   56642
6               3.0   60150
7               3.2   54445
8               3.2   64445
9               3.7   57189
10              3.9   63218
11              4.0   55794
12              4.0   56957
13              4.1   57081
14              4.5   61111
15              4.9   67938
16              5.1   66029
17              5.3   83088
18              5.9   81363
19              6.0   93940
20              6.8   91738
21              7.1   98273
22              7.9  101302
23              8.2  113812
24              8.7  109431
25              9.0  105582
```

```
23              8.2  113812
24              8.7  109431
25              9.0  105582
26              9.5  116969
27              9.6  112635
28             10.3  122391
29             10.5  121872
```

[7]:
```python
# Statistical summary of the data using describe()
descreption = df.describe()
print(descreption) # or
print(df.describe())
```

```
       YearsExperience         Salary
count        30.000000      30.000000
mean          5.313333   76003.000000
std           2.837888   27414.429785
min           1.100000   37731.000000
25%           3.200000   56720.750000
50%           4.700000   65237.000000
75%           7.700000  100544.750000
max          10.500000  122391.000000
       YearsExperience         Salary
count        30.000000      30.000000
mean          5.313333   76003.000000
std           2.837888   27414.429785
min           1.100000   37731.000000
25%           3.200000   56720.750000
50%           4.700000   65237.000000
75%           7.700000  100544.750000
max          10.500000  122391.000000
```

[8]:
```
# Calculating quantiles
quantiles = df['YearsExperience'].quantile([0.25, 0.5, 0.75])
print(quantiles)
```

```
0.25    3.2
0.50    4.7
0.75    7.7
Name: YearsExperience, dtype: float64
```

[9]:
```
import pandas as pd

data = [10, 20, 20, 30, 100]
print(pd.Series(data).skew())
```

```
2.0286991020803327
```

[10]:
```
# Calculating skewness and kurtosis
skewness = df['YearsExperience'].skew()
print(skewness )
kurtosis = df['YearsExperience'].kurt()
print(kurtosis)
```

```
0.37956024064804106
-1.0122119403325072
```

[11]:
```python
# Calculating value counts for unique values in the 'YearsExperience' column
value_counts = df['YearsExperience'].value_counts()
print(value_counts)
df.YearsExperience.value_counts()     # (or)
```

```
YearsExperience
3.2     2
4.0     2
1.1     1
5.3     1
10.3    1
9.6     1
9.5     1
9.0     1
8.7     1
8.2     1
7.9     1
7.1     1
6.8     1
6.0     1
5.9     1
5.1     1
1.3     1
4.9     1
4.5     1
4.1     1
3.9     1
3.7     1
3.0     1
2.9     1
2.2     1
2.0     1
1.5     1
10.5    1
```

```
2.0     1
1.5     1
10.5    1
Name: count, dtype: int64
```

[11]:
```
YearsExperience
3.2     2
4.0     2
1.1     1
5.3     1
10.3    1
9.6     1
9.5     1
9.0     1
8.7     1
8.2     1
7.9     1
7.1     1
6.8     1
6.0     1
5.9     1
5.1     1
1.3     1
4.9     1
4.5     1
4.1     1
3.9     1
3.7     1
3.0     1
2.9     1
2.2     1
2.0     1
1.5     1
10.5    1
Name: count, dtype: int64
```

```
[3]: import pandas as pd
df = pd.read_csv("C:\\Users\\RVUW280\\Downloads\\Laptops - Sheet1.csv")
print(df)
pd.get_dummies(df)

from sklearn.preprocessing import OneHotEncoder
enc = OneHotEncoder()
enc_data = enc.fit_transform(df[['Category']])
print(enc_data)

print(enc.categories_ )
enc_data.toarray()
enc_df = pd.DataFrame(enc_data.toarray())
print(enc_df)
enc_df = pd.DataFrame(enc_data.toarray(), columns = ['2 in 1 Convertible',
'Gaming', 'Netbook', 'Notebook', 'Ultrabook', 'Workstation'])
print(enc_df)
df1 = df.join(enc_df)
print(df1)
```

```
    Manufacturer       Model_Name   Category  Screen_size_inches  \
0          Apple      MacBook Pro  Ultrabook                13.3
1          Apple      Macbook Air  Ultrabook                13.3
2             HP           250 G6   Notebook                15.6
3          Apple      MacBook Pro  Ultrabook                15.4
4          Apple      MacBook Pro  Ultrabook                13.3
..           ...              ...        ...                 ...
972         Dell     Alienware 17     Gaming                17.3
973      Toshiba  Tecra A40-C-1DF   Notebook                14.0
974         Asus        Rog Strix     Gaming                17.3
975           HP      Probook 450   Notebook                15.6
976       Lenovo    ThinkPad T460   Notebook                14.0

                                   Screen                          CPU   RAM  \
```

```
                                 Screen                          CPU   RAM  \
0    IPS Panel Retina Display 2560x1600         Intel Core i5 2.3GHz   8GB
1                             1440x900         Intel Core i5 1.8GHz   8GB
2                     Full HD 1920x1080   Intel Core i5 7200U 2.5GHz   8GB
3    IPS Panel Retina Display 2880x1800         Intel Core i7 2.7GHz  16GB
```

[4]:
```python
# Using label encoding
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()

# Fit and transform the data
df['RAM'] = le.fit_transform(df['RAM'])
print(df)

print(le.classes_ )
```

```
    Manufacturer       Model_Name   Category  Screen_size_inches  \
0          Apple      MacBook Pro  Ultrabook                13.3
1          Apple      Macbook Air  Ultrabook                13.3
2             HP           250 G6   Notebook                15.6
3          Apple      MacBook Pro  Ultrabook                15.4
4          Apple      MacBook Pro  Ultrabook                13.3
..           ...              ...        ...                 ...
972         Dell     Alienware 17     Gaming                17.3
973      Toshiba  Tecra A40-C-1DF   Notebook                14.0
974         Asus        Rog Strix     Gaming                17.3
975           HP      Probook 450   Notebook                15.6
976       Lenovo    ThinkPad T460   Notebook                14.0

                                 Screen                          CPU  RAM  \
0    IPS Panel Retina Display 2560x1600         Intel Core i5 2.3GHz    7
1                             1440x900         Intel Core i5 1.8GHz    7
2                     Full HD 1920x1080   Intel Core i5 7200U 2.5GHz    7
3    IPS Panel Retina Display 2880x1800         Intel Core i7 2.7GHz    1
4    IPS Panel Retina Display 2560x1600         Intel Core i5 3.1GHz    7
```

```
                  Storage                           GPU Operating_System  \
0               128GB SSD  Intel Iris Plus Graphics 640            macOS
1     128GB Flash Storage        Intel HD Graphics 6000            macOS
2               256GB SSD         Intel HD Graphics 620            No OS
3               512GB SSD            AMD Radeon Pro 455            macOS
4               256GB SSD  Intel Iris Plus Graphics 650            macOS
..                    ...                            ...              ...
972  256GB SSD +  1TB HDD       Nvidia GeForce GTX 1070          Windows
973             256GB SSD         Intel HD Graphics 520          Windows
974  256GB SSD +  1TB HDD       Nvidia GeForce GTX 1060          Windows
975  128GB SSD +  1TB HDD          Nvidia GeForce 930MX          Windows
976          508GB Hybrid         Intel HD Graphics 520          Windows

    Operating_System_Version  Weight_kg        Price
0                        NaN       1.37  11912523.48
1                        NaN       1.34   7993374.48
2                        NaN       1.86   5112900.00
3                        NaN       1.83  22563005.40
4                        NaN       1.37  16037611.20
..                       ...        ...          ...
972                       10       4.42  24897600.00
973                       10       1.95  10492560.00
974                       10       2.73  18227710.80
975                       10       2.04   8705268.00
976                        7       1.70   8909784.00

[977 rows x 13 columns]
['12GB' '16GB' '24GB' '2GB' '32GB' '4GB' '6GB' '8GB']
```

[5]:
```python
import pandas as pd
data=[1,2,3]
df = pd.DataFrame(data)
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
scaled_data = scaler.fit_transform(df)

print(scaled_data)

from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
scaled_data = scaler.fit_transform(df)

print(scaled_data)

from sklearn.preprocessing import Normalizer

scaler = Normalizer()
normalized_data = scaler.fit_transform(df)

print(normalized_data)
```

```
[[-1.22474487]
 [ 0.        ]
 [ 1.22474487]]
[[0. ]
 [0.5]
 [1. ]]
[[1.]
 [1.]
 [1.]]
```

[6]:
```python
import pandas as pd
data=pd.read_csv("C:\\Users\\RVUW280\\Downloads\\experience  - Sheet1.csv")
df = pd.DataFrame(data)
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
scaled_data = scaler.fit_transform(df)

print(scaled_data)

print (" ")
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
scaled_data = scaler.fit_transform(df)

print(scaled_data)
print (" ")
from sklearn.preprocessing import Normalizer

scaler = Normalizer()
normalized_data = scaler.fit_transform(df)

print(normalized_data)
```

```
[[-1.51005294 -1.36011263]
 [-1.43837321 -1.10552744]
 [-1.36669348 -1.419919  ]
 [-1.18749416 -1.20495739]
 [-1.11581443 -1.33978143]
 [-0.86493538 -0.71830716]
 [-0.82909552 -0.58815781]
 [-0.75741579 -0.79981746]
 [-0.75741579 -0.42881019]
```

```
 [-0.57821647 -0.69801306]
 [-0.50653674 -0.47433279]
 [-0.47069688 -0.74976858]
 [-0.47069688 -0.70662043]
 [-0.43485702 -0.70201994]
 [-0.29149756 -0.55250402]
 [-0.1481381  -0.29921736]
 [-0.07645838 -0.37004264]
 [-0.00477865  0.26285865]
 [ 0.21026054  0.19885989]
 [ 0.2461004   0.66547573]
 [ 0.53281931  0.58377993]
 [ 0.6403389   0.82623317]
 [ 0.92705781  0.93861127]
 [ 1.03457741  1.40274136]
 [ 1.21377673  1.24020308]
 [ 1.32129632  1.09740238]
 [ 1.50049564  1.51986835]
 [ 1.5363355   1.3590738 ]
 [ 1.78721455  1.72102849]
 [ 1.85889428  1.70177321]]

[[0.         0.01904087]
 [0.0212766  0.1000945 ]
 [0.04255319 0.        ]
 [0.09574468 0.06843846]
 [0.11702128 0.02551382]
 [0.19148936 0.22337586]
 [0.20212766 0.26481219]
 [0.22340426 0.19742499]
 [0.22340426 0.31554453]
 [0.27659574 0.229837  ]
```

```
 [0.29787234 0.30105126]
 [0.30851064 0.21335932]
 [0.30851064 0.22709662]
 [0.31914894 0.2285613 ]
 [0.36170213 0.27616348]
 [0.40425532 0.35680369]
 [0.42553191 0.33425467]
 [0.44680851 0.53575478]
 [0.5106383  0.51537916]
 [0.5212766  0.66393811]
 [0.60638298 0.63792818]
 [0.63829787 0.7151193 ]
 [0.72340426 0.75089771]
 [0.75531915 0.89866525]
 [0.80851064 0.84691708]
 [0.84042553 0.80145287]
 [0.89361702 0.93595559]
 [0.90425532 0.88476258]
 [0.9787234  1.        ]
 [1.         0.9938696 ]]

[[2.79592303e-05 1.00000000e+00]
 [2.81354832e-05 1.00000000e+00]
 [3.97551085e-05 9.99999999e-01]
 [4.59506031e-05 9.99999999e-01]
 [5.51502844e-05 9.99999998e-01]
 [5.11987570e-05 9.99999999e-01]
 [4.98753117e-05 9.99999999e-01]
 [5.87749104e-05 9.99999998e-01]
 [4.96547443e-05 9.99999999e-01]
 [6.46977564e-05 9.99999998e-01]
 [6.16912903e-05 9.99999998e-01]
 [7.16922965e-05 9.99999997e-01]
 [7.02284178e-05 9.99999998e-01]
 [7.18277534e-05 9.99999997e-01]
```

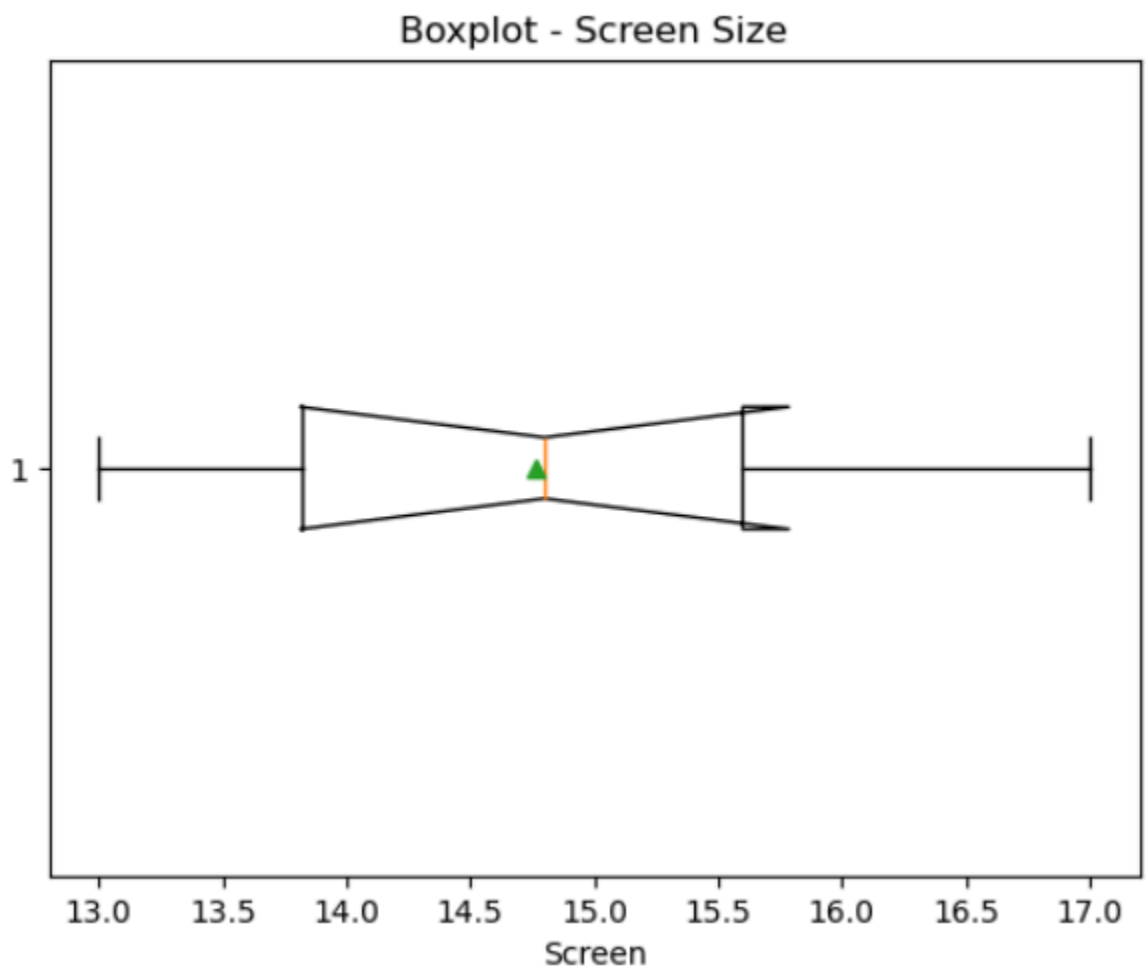
```
[4.98753117e-05 9.99999999e-01]
[5.87749104e-05 9.99999998e-01]
[4.96547443e-05 9.99999999e-01]
[6.46977564e-05 9.99999998e-01]
[6.16912903e-05 9.99999998e-01]
[7.16922965e-05 9.99999997e-01]
[7.02284178e-05 9.99999998e-01]
[7.18277534e-05 9.99999997e-01]
[7.36364973e-05 9.99999997e-01]
[7.21245840e-05 9.99999997e-01]
[7.72387888e-05 9.99999997e-01]
[6.37877911e-05 9.99999998e-01]
[7.25145334e-05 9.99999997e-01]
[6.38705555e-05 9.99999998e-01]
[7.41241359e-05 9.99999997e-01]
[7.22477179e-05 9.99999997e-01]
[7.79846398e-05 9.99999997e-01]
[7.20486414e-05 9.99999997e-01]
[7.95021518e-05 9.99999997e-01]
[8.52418023e-05 9.99999996e-01]
[8.12181003e-05 9.99999997e-01]
[8.52310558e-05 9.99999996e-01]
[8.41565145e-05 9.99999996e-01]
[8.61559666e-05 9.99999996e-01]]
```

[6]:
```python
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Load dataset
df = pd.read_csv("C:\\Users\\RVUW280\\Downloads\\LAPTOPS - Sheet1 (1).csv")

print("Dataset:")
print(df)


# Matplotlib Boxplot - Screen

print("\nBoxplot for Screen Size")

plt.boxplot(df["Screen"], notch=True, vert=False, showmeans=True)
plt.xlabel("Screen")
plt.title("Boxplot - Screen Size")
plt.show()


# Matplotlib Boxplot - Weight
print("\nBoxplot for Weight")

plt.boxplot(df["Weight"])
plt.ylabel("Weight")
plt.title("Boxplot - Weight")
plt.show()
```

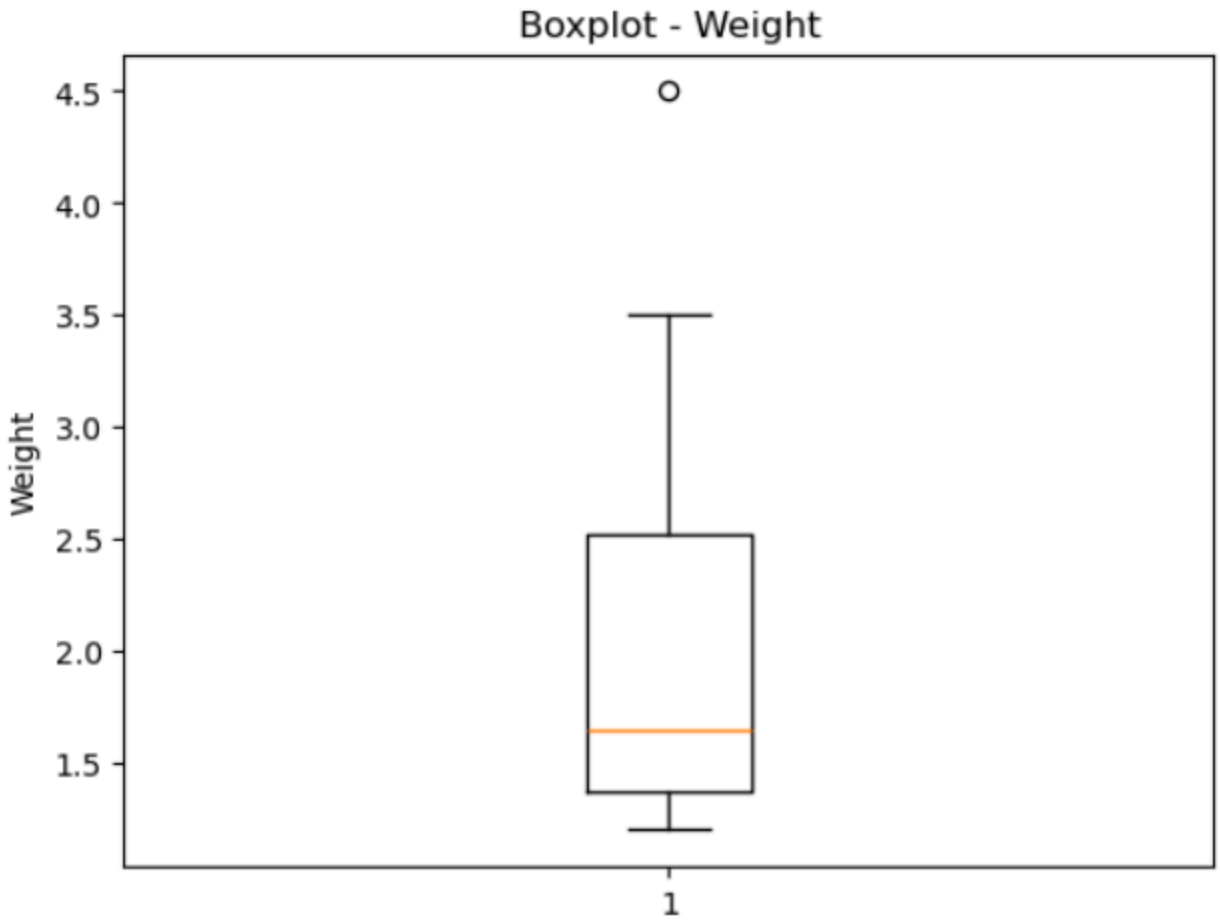
```python
# Seaborn Boxplot
print("\nSeaborn Boxplot for Weight")

sns.boxplot(x=df["Weight"])
plt.title("Seaborn Boxplot - Weight")
plt.show()

print("\nSeaborn Boxplot - Weight vs RAM")

sns.boxplot(x=df["Weight"], y=df["RAM"])
plt.title("Weight vs RAM")
plt.show()


# IQR Method
print("\nIQR Method Calculation")

Q1 = df["Weight"].quantile(0.25)
Q3 = df["Weight"].quantile(0.75)

print("Q1:", Q1)
print("Q3:", Q3)

IQR = Q3 - Q1
print("IQR:", IQR)

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

print("Lower Bound:", lower)
print("Upper Bound:", upper)
```

```python
# Outliers
outliers = df[(df["Weight"] < lower) | (df["Weight"] > upper)]

print("\nOutliers in Weight:")
print(outliers)
```

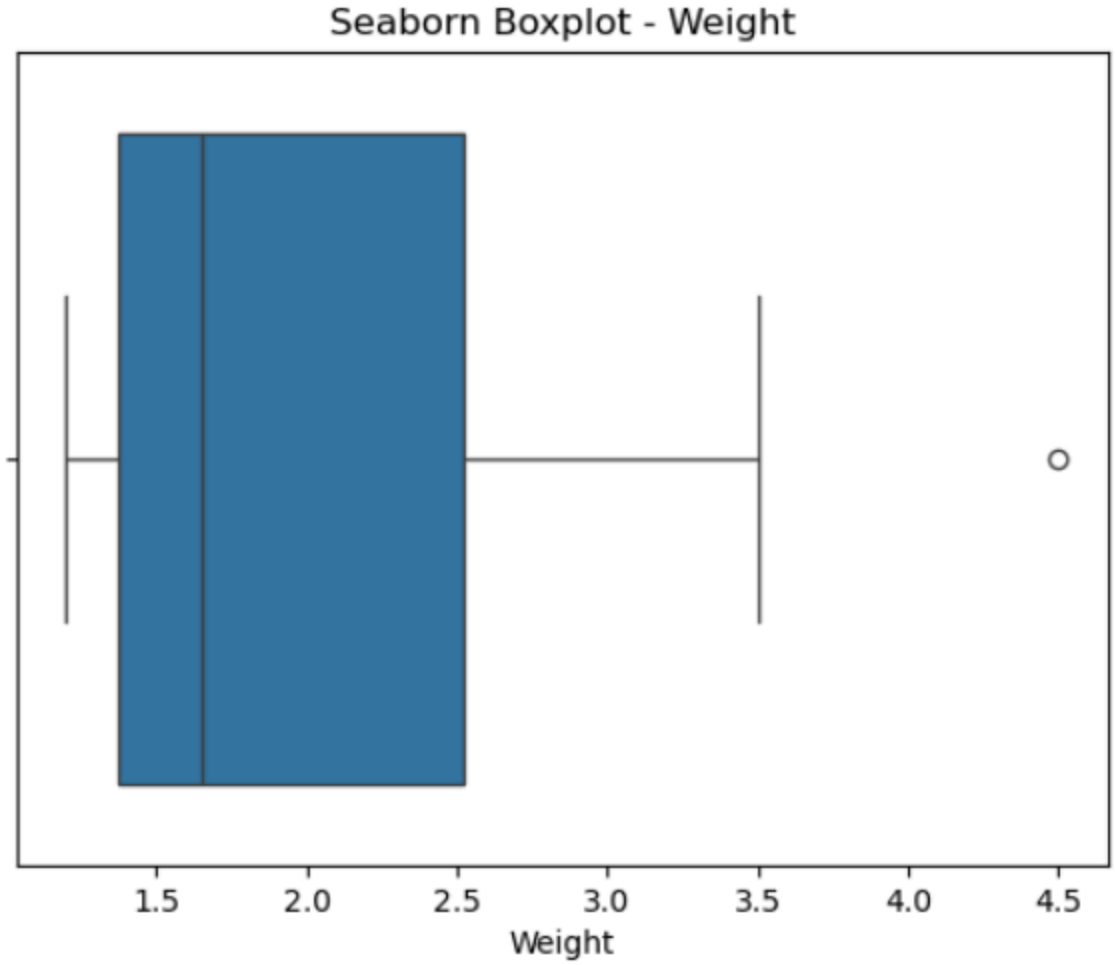
```
Dataset:
      Name  Screen  Weight  RAM   Price
0     Dell    15.6     1.8    8   55000
1       HP    14.0     1.5    8   50000
2   Lenovo    15.6     2.2   16   65000
3     Asus    13.3     1.3    8   48000
4     Acer    17.0     3.5   16   70000
5    Apple    13.0     1.2    8   90000
6      MSI    15.6     4.5   32  120000
7  Samsung    14.0     1.4    8   52000

 Boxplot for Screen Size
```



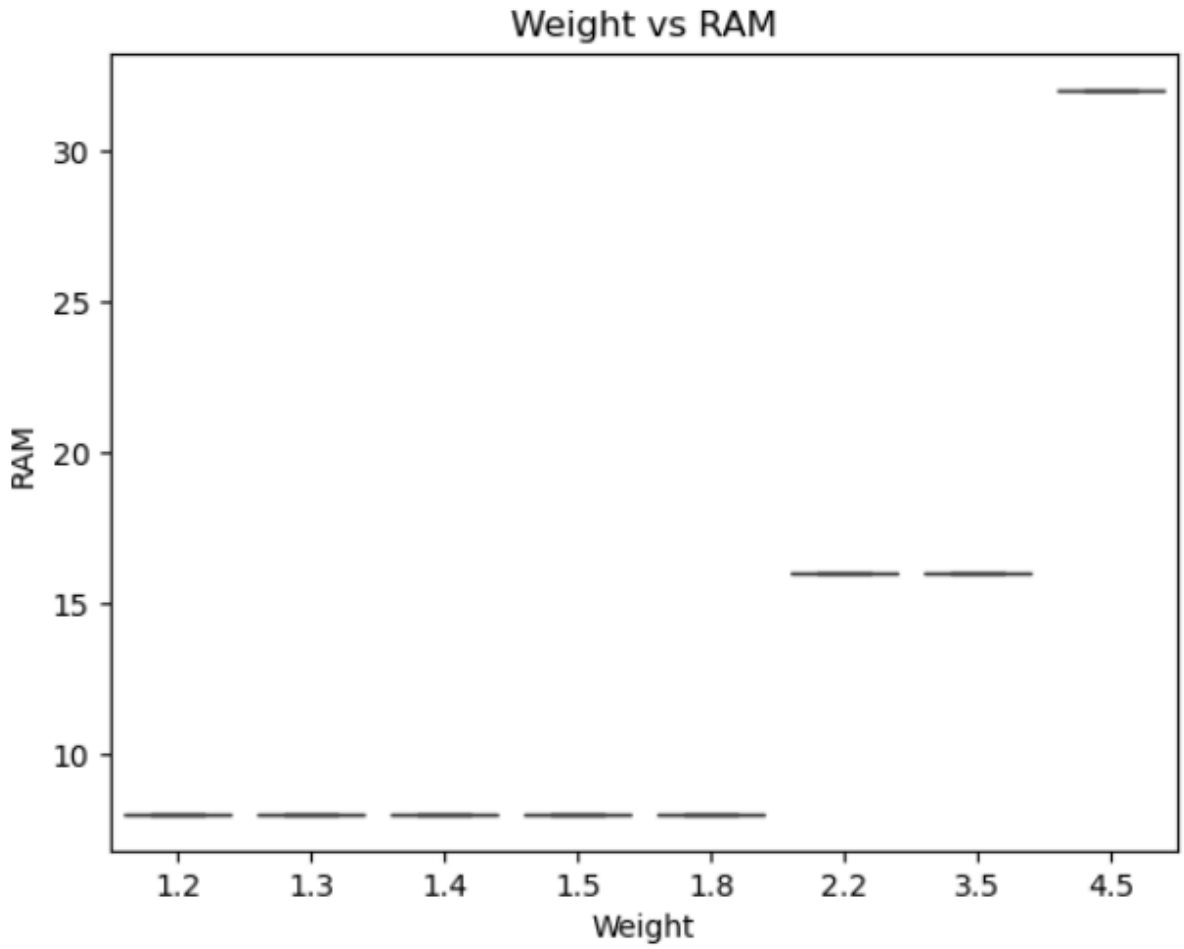
```
Boxplot for Weight
```



```
Seaborn Boxplot for Weight
```



```
Seaborn Boxplot - Weight vs RAM
```




```
IQR Method Calculation
Q1: 1.375
Q3: 2.5250000000000004
IQR: 1.1500000000000004
Lower Bound: -0.35000000000000053
Upper Bound: 4.250000000000001

Outliers in Weight:
  Name  Screen  Weight  RAM   Price
6  MSI    15.6     4.5   32  120000
```

[2]:

```python
import pandas as pd

emp = pd.DataFrame({
    'ID': [101, 102, 103, 104, 105],
    'Name': ['A', 'B', 'C', 'D', 'E'],
    'Dept': ['HR', 'Eng', 'Eng', 'HR', 'Mkt']
})

sal = pd.DataFrame({
    'ID': [101, 102, 103, 104, 105],
    'Sal': [50000, 70000, 80000, 55000, 60000]
})

sales1 = pd.DataFrame({
    'Date': pd.date_range(start='2026-01-01', periods=5),
    'Reg': ['N']*5,  # same as ['N','N','N','N','N']
    'Sale': [250, 300, 200, 400, 350]
})

sales2 = pd.DataFrame({
    'Date': pd.date_range(start='2026-01-01', periods=5),
    'Reg': ['S']*5,
    'Sale': [300, 320, 280, 360, 310]
})

print("Employee Data:")
print(emp)

print("\nSalary Data:")
print(sal)
```

```python
# Merging

merge_data = pd.merge(emp, sal, on='ID')

print("\nMerged Data:")
print(merge_data)

# GroupBy

avg = emp.merge(sal, on='ID').groupby('Dept')['Sal'].mean() .reset_index()
avg.columns=['dept','avgsal']

print("\nAverage Salary by Dept:")
print(avg)

# Joining

emp1 = emp.set_index('ID')
sal1 = sal.set_index('ID')
join_data = emp1.join(sal1,how='inner')

print("\nJoined Data:")
print(join_data)

# Concatenation

final = pd.concat([sales1, sales2], axis=1)
print("\nConcatenated Sales Data:")
print(final)
```

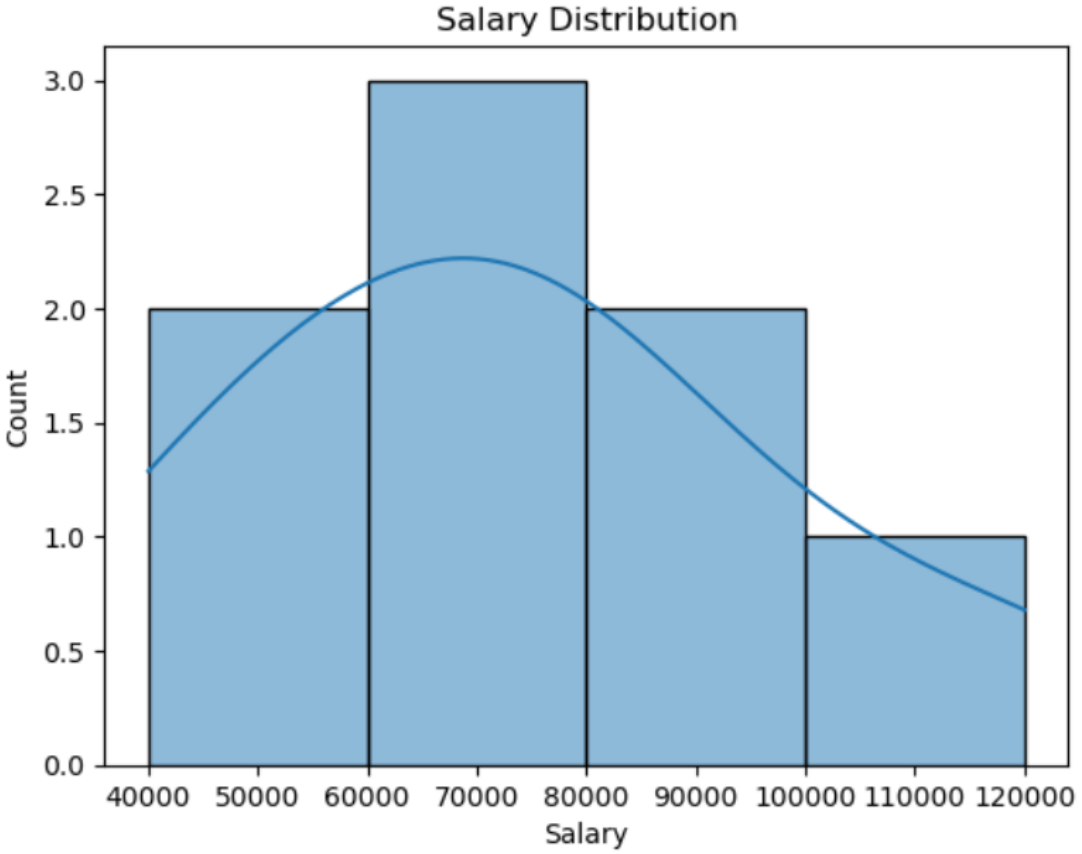
```
Employee Data:
    ID Name Dept
0  101    A   HR
1  102    B  Eng
2  103    C  Eng
3  104    D   HR
4  105    E  Mkt

Salary Data:
    ID    Sal
0  101  50000
1  102  70000
2  103  80000
3  104  55000
4  105  60000

Merged Data:
    ID Name Dept    Sal
0  101    A   HR  50000
1  102    B  Eng  70000
2  103    C  Eng  80000
3  104    D   HR  55000
4  105    E  Mkt  60000

Average Salary by Dept:
  dept   avgsal
0  Eng  75000.0
1   HR  52500.0
2  Mkt  60000.0
```

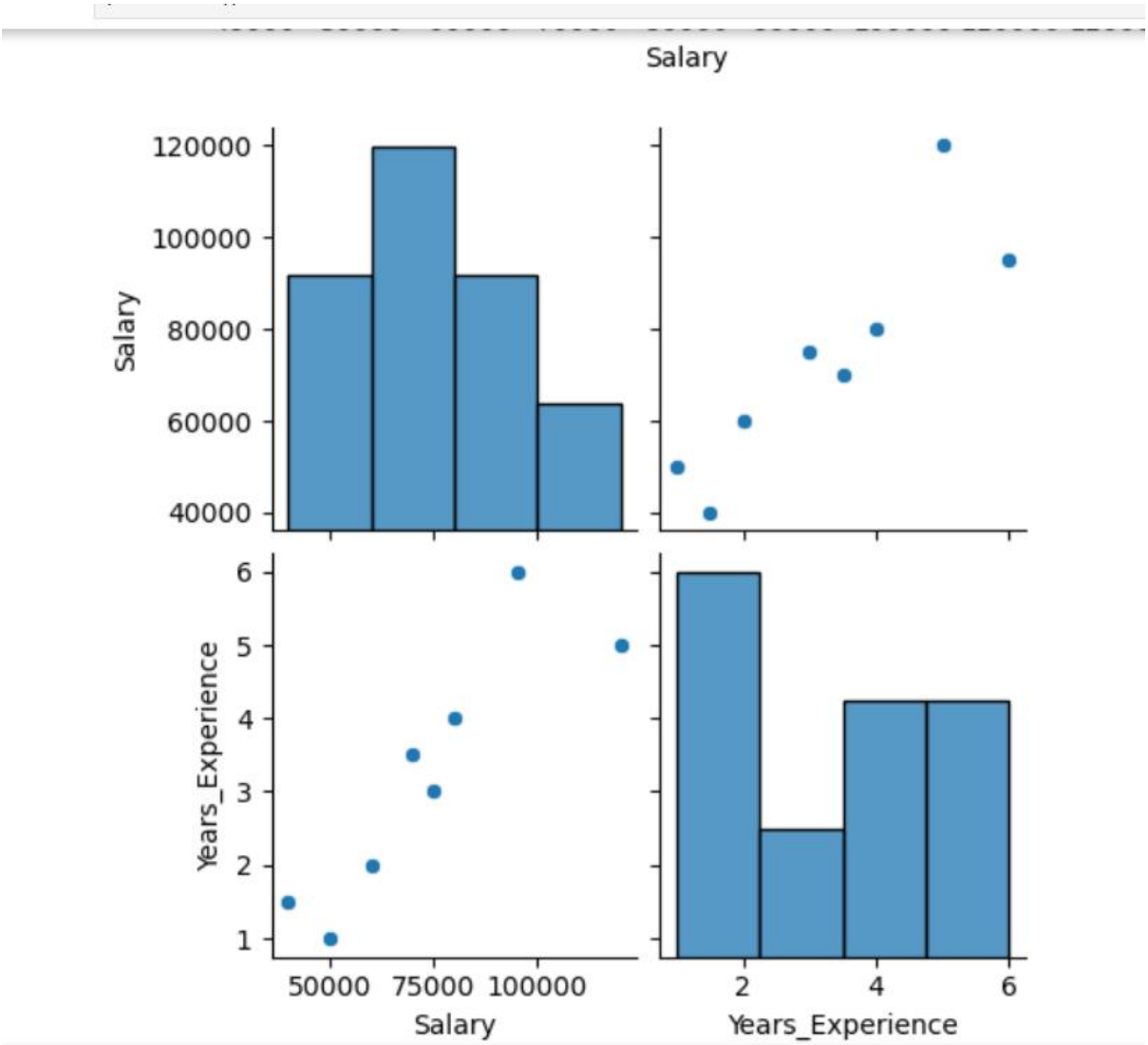
```
Joined Data:
    Name Dept    Sal
ID
101    A   HR  50000
102    B  Eng  70000
103    C  Eng  80000
104    D   HR  55000
105    E  Mkt  60000

Concatenated Sales Data:
         Date Reg  Sale       Date Reg  Sale
0 2026-01-01   N   250 2026-01-01   S   300
1 2026-01-02   N   300 2026-01-02   S   320
2 2026-01-03   N   200 2026-01-03   S   280
3 2026-01-04   N   400 2026-01-04   S   360
4 2026-01-05   N   350 2026-01-05   S   310
```

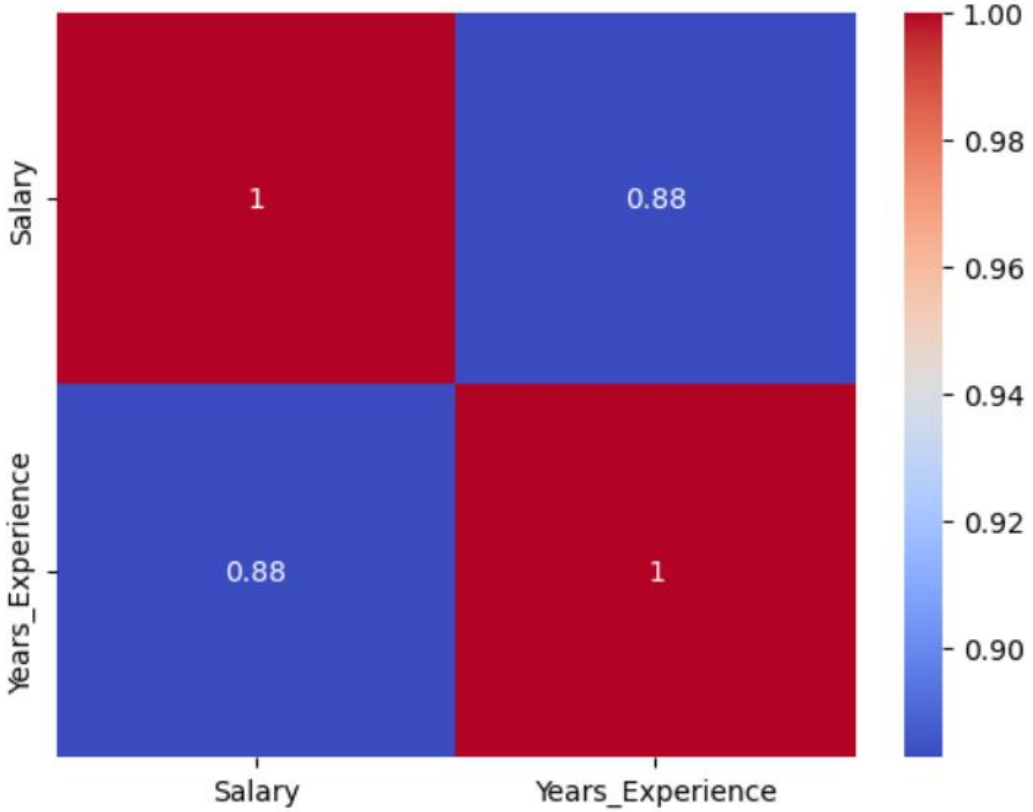
```python
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

data = {
'Salary': [50000, 60000, 75000, 80000, 120000, 95000, 40000, 70000],
'Years_Experience': [1, 2, 3, 4, 5, 6, 1.5, 3.5],
'Experience_Level': ['Junior', 'Junior', 'Mid', 'Mid', 'Senior', 'Senior', 'Junior', 'Mid']
}
df = pd.DataFrame(data)

sns.histplot(df['Salary'], kde=True)
plt.title("Salary Distribution")
plt.show()

sns.pairplot(df[['Salary', 'Years_Experience']])
plt.show()

sns.heatmap(df[['Salary', 'Years_Experience']].corr(), annot=True, cmap='coolwarm')
plt.show()

sns.lineplot(x='Years_Experience', y='Salary', data=df, marker='o')

plt.title("Salary Trend by Experience")
plt.show()

sns.scatterplot(x='Years_Experience', y='Salary', data=df, s=100)

plt.title("Salary vs Experience")
plt.show()
```


```python
plt.title("Salary Trend by Experience")
plt.show()

sns.scatterplot(x='Years_Experience', y='Salary', data=df, s=100)

plt.title("Salary vs Experience")
plt.show()
```

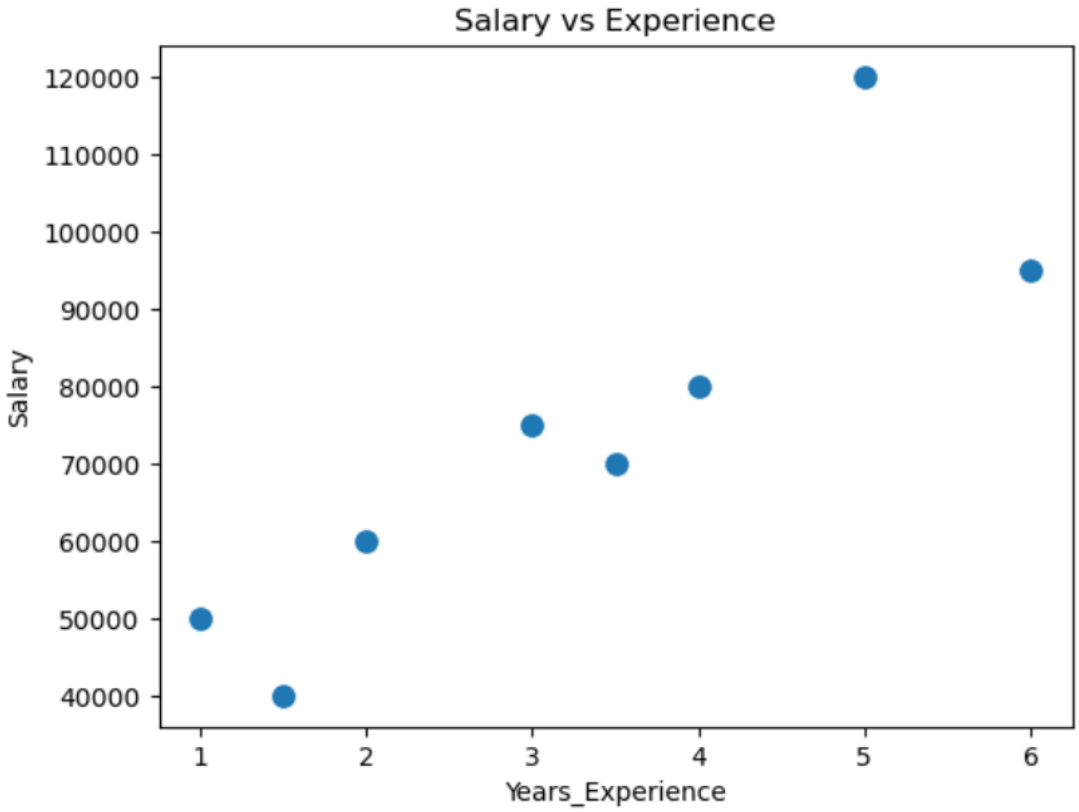

```python
sns.barplot(x='Experience_Level', y='Salary', data=df)
plt.title("Salary by Experience Level")
plt.show()
```


Salary Distribution
3.0
2.5
2.0
1.5
1.0
0.5
0.0
Count
40000
50000
60000
70000
80000
90000
100000
110000
120000
Salary


Salary
120000
100000
80000
60000
40000
Salary
6
5
4
3
2
1
Years_Experience
50000
75000
100000
Salary
2
4
6
Years_Experience


Salary
Years_Experience
1
0.88
0.88
1
Salary
Years_Experience
1.00
0.98
0.96
0.94
0.92
0.90

Salary Trend by Experience
Salary
120000
110000
100000
90000
80000
70000
60000
50000
40000
1
2
3
4
5
6
Years_Experience



Salary by Experience Level

Salary by Experience Level
120000
100000
80000
60000
40000
20000
0
Salary
Junior
Mid
Senior
Experience_Level